COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S4
DATE PERFORMED:	17/07/2026	**DATE SUBMITTED:**	17/07/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	Engr.Jim Lord Quejado

ACTIVITY NO. 2.1 Arrays, Pointers, and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOA & ILOB

Screenshot

hands on activity 2 > ACTWO.cpp > Student > ~Student()

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
    public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};
int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

```
stderr=Microsoft-MIEngine-Error-rjikyfb5.se0' '--pid=Micr
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Documents\Almirol joel cpes4>
```

Observation	The sample code written in C++ illustrates object's lifecycle through various constructors and destructors. On the declaration of objects student1 and student3, the parameterized constructor is called ("Constructor Called"), whereas when the object student2 is initialized using student1, the custom copy constructor is invoked ("Copy Constructor Called"). Assignment of student3 = student2 doesn't output anything since it invokes the compiler's default assignment operator, not the constructor. In addition, upon exit of main(), the destruction of all three objects takes place ("Destructor Called") in reverse sequence of their creation.

Table 2-1. Initial Driver Program

Screenshot	

```
hands on activity 2 > ACTIVO.cpp > main()
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;
    public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    //Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }

};
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j; i++) { //loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

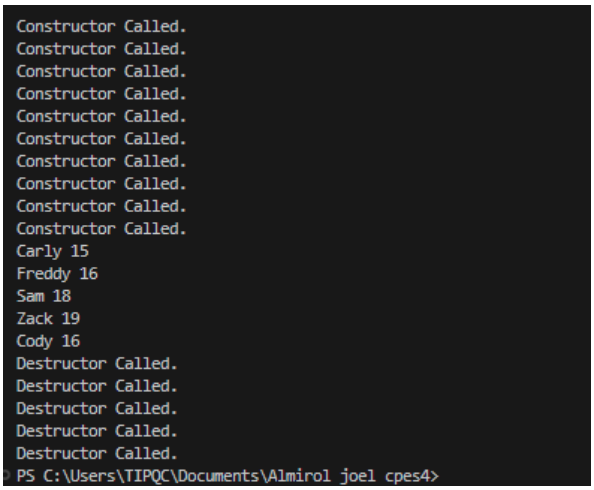
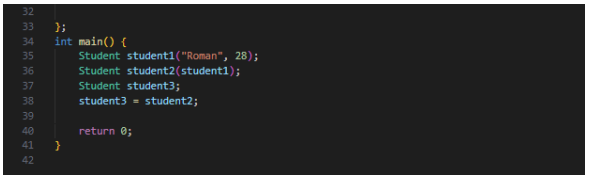
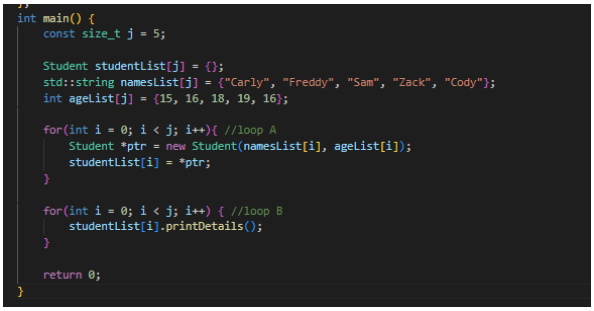
	
Observation	The modified code illustrates the concept of lifecycle of objects by combining an array along with dynamically allocated memory. Firstly, declaring the fixed array studentList[j] creates five default Student objects, calling the constructor five times. Secondly, within the first loop, five more Student objects are created using the new keyword, Student(...). This causes "Constructor Called." to be printed five times, making the total number of times the constructor is called as ten until their data is copied using assignment. In the third loop, all that happens is the invocation of printDetails() method to print names and ages. Lastly, at the termination of main(), only the five objects statically created in studentList are destroyed (five times printing "Destructor Called.") whereas the five objects created dynamically using new are stuck in memory without any delete command being used.

Table 2-2. Modified Driver Program with Student Lists

Loop A	

Observation	The above code creates an object of type Student using the parameterized constructor. In the above line, Student student2(student1); the Copy Constructor is called explicitly to create a duplicate object. Student student3; creates an object by calling the default constructor. student3 = student2; uses the default assignment operator (=).
Loop B	
Observation	Array & Dynamic Allocation: In Loop B, the array is statically allocated while the pointer is dynamically instantiated. The declaration Student studentList[5] will initialize the default object of Student. The loop iterates through the namesList and ageList arrays and allocates a temporary object dynamically with the pointer Student *ptr = new Student(...). This is assigned with the help of (*ptr).

```
int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for(int i = 0; i < j; i++) { //loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

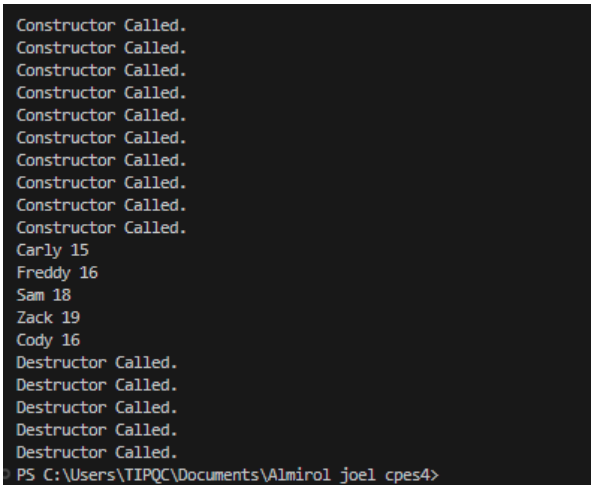
Output	
Observation	Redundant Allocations: The result is the same as the starting driver program. The sequential process of construction begins with 10 constructors, followed by the iteration and printing of students using .printDetails() and ends up with 5 destructors. This indicates the problem of value copying done by studentList[i] = *ptr;, which does not deallocate the heap memory.

Table 2-3. Final Driver Program

Modifications	
Observation	

Supplementary Activity

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenna's Grocery List

Apple	**PHP 10**	**x7**
Banana	**PHP 10**	**x8**
Broccoli	**PHP 60**	**x12**

Lettuce	PHP 50	x10


```cpp
#include <iostream>
#include <string>
#include <utility> // For std::move

// Common Abstract Base Class to allow storing both Fruits and Vegetables in one array
class GroceryItem {
protected:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    GroceryItem(std::string name = "Unknown", double price = 0.0, int quantity = 0)
        : name(std::move(name)), price(price), quantity(quantity) {}

    // Virtual Destructor (Crucial for base class pointers to deallocate memory correctly)
    virtual ~GroceryItem() {
        std::cout << "[Destructor] GroceryItem base class cleaned up." << std::endl;
    }

    // Copy Constructor
    GroceryItem(const GroceryItem& other)
        : name(other.name), price(other.price), quantity(other.quantity) {
        std::cout << "[Copy Constructor] GroceryItem copied." << std::endl;
    }

    // Copy Assignment Operator
    GroceryItem& operator=(const GroceryItem& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
            std::cout << "[Copy Assignment] GroceryItem assigned." << std::endl;
        }
        return *this;
    }

    // Helper functions
    virtual double calculateSum() const {
        return price * quantity;
    }

    virtual std::string getType() const = 0; // Pure virtual function to distinguish types

    virtual void displayDetails() const {
        std::cout << "Type: " << getType()
                  << " | Name: " << name
                  << " | Price: PHP " << price
                  << " | Qty: x" << quantity
                  << " | Subtotal: PHP " << calculateSum() << std::endl;
    }
```

```cpp
    std::string getName() const { return name; }
};

// --- PROBLEM 1: Derived Classes with Rule of Three ---

class Fruit : public GroceryItem {
public:
    Fruit(std::string name = "Unknown Fruit", double price = 0.0, int quantity = 0)
        : GroceryItem(std::move(name), price, quantity) {
        std::cout << "[Constructor] Fruit: " << this->name << " created." << std::endl;
    }

    ~Fruit() override {
        std::cout << "[Destructor] Fruit: " << name << " destroyed." << std::endl;
    }

    Fruit(const Fruit& other) : GroceryItem(other) {
        std::cout << "[Copy Constructor] Fruit: " << name << " copied." << std::endl;
    }

    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            GroceryItem::operator=(other);
            std::cout << "[Copy Assignment] Fruit: " << name << " assigned." << std::endl;
        }
        return *this;
    }

    std::string getType() const override {
        return "Fruit";
    }
};

class Vegetable : public GroceryItem {
public:
    Vegetable(std::string name = "Unknown Vegetable", double price = 0.0, int quantity = 0)
        : GroceryItem(std::move(name), price, quantity) {
        std::cout << "[Constructor] Vegetable: " << this->name << " created." << std::endl;
    }

    ~Vegetable() override {
        std::cout << "[Destructor] Vegetable: " << name << " destroyed." << std::endl;
    }

    Vegetable(const Vegetable& other) : GroceryItem(other) {
        std::cout << "[Copy Constructor] Vegetable: " << name << " copied." << std::endl;
    }

    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
            GroceryItem::operator=(other);
            std::cout << "[Copy Assignment] Vegetable: " << name << " assigned." << std::endl;
        }
        return *this;
    }

```

```cpp
    std::string getType() const override {
        return "Vegetable";
    }
};

// --- PROBLEM 3: TotalSum Calculation Function ---
double calculateTotalSum(GroceryItem* groceryList[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; ++i) {
        if (groceryList[i] != nullptr) {
            total += groceryList[i]->calculateSum();
        }
    }
    return total;
}

// --- DRIVER CODE ---
int main() {
    const int LIST_SIZE = 4;

    // --- PROBLEM 2: GroceryList Array with Dynamic Allocation ---
    std::cout << "=== INITIALIZING JENNA'S GROCERY LIST ===" << std::endl;
    GroceryItem* groceryList[LIST_SIZE];

    groceryList[0] = new Fruit("Apple", 10.0, 7);
    groceryList[1] = new Fruit("Banana", 10.0, 8);
    groceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    groceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    std::cout << "\n=== DISPLAYING GROCERY LIST DETAILS ===" << std::endl;
    for (int i = 0; i < LIST_SIZE; ++i) {
        if (groceryList[i] != nullptr) {
            groceryList[i]->displayDetails();
        }
    }

    std::cout << "\n=== CALCULATING TOTAL SUM ===" << std::endl;
    double initialTotal = calculateTotalSum(groceryList, LIST_SIZE);
    std::cout << "Total Bill: PHP " << initialTotal << std::endl;

    // --- PROBLEM 4: Deleting Lettuce and De-allocating Memory ---
    std::cout << "\n=== REMOVING LETTUCE FROM THE LIST ===" << std::endl;
    for (int i = 0; i < LIST_SIZE; ++i) {
        if (groceryList[i] != nullptr && groceryList[i]->getName() == "Lettuce") {
            delete groceryList[i];      // Safely deallocate heap memory
            groceryList[i] = nullptr;   // Prevent wild/dangling pointers
            std::cout << "Lettuce successfully deleted and memory de-allocated!" << std::endl;
            break;
        }
    }

    std::cout << "\n=== UPDATED BILL AFTER REMOVAL ===" << std::endl;
    double updatedTotal = calculateTotalSum(groceryList, LIST_SIZE);
    std::cout << "New Total Bill: PHP " << updatedTotal << std::endl;

    // --- CLEANUP (Deallocating remaining dynamic memory before program ends) ---
```

```cpp
    // --- CLEANUP (Deallocating remaining dynamic memory before program ends) ---
    std::cout << "\n=== CLEANING UP REMAINING ITEMS ===" << std::endl;
    for (int i = 0; i < LIST_SIZE; ++i) {
        if (groceryList[i] != nullptr) {
            delete groceryList[i];
            groceryList[i] = nullptr;
        }
    }

    return 0;
}
```

```
=== INITIALIZING JENNA'S GROCERY LIST ===
[Constructor] Fruit: Apple created.
[Constructor] Fruit: Banana created.
[Constructor] Vegetable: Broccoli created.
[Constructor] Vegetable: Lettuce created.

=== DISPLAYING GROCERY LIST DETAILS ===
Type: Fruit | Name: Apple | Price: PHP 10 | Qty: x7 | Subtotal: PHP 70
Type: Fruit | Name: Banana | Price: PHP 10 | Qty: x8 | Subtotal: PHP 80
Type: Vegetable | Name: Broccoli | Price: PHP 60 | Qty: x12 | Subtotal: PHP 720
Type: Vegetable | Name: Lettuce | Price: PHP 50 | Qty: x10 | Subtotal: PHP 500

=== CALCULATING TOTAL SUM ===
Total Bill: PHP 1370

=== REMOVING LETTUCE FROM THE LIST ===
[Destructor] Vegetable: Lettuce destroyed.
[Destructor] GroceryItem base class cleaned up.
Lettuce successfully deleted and memory de-allocated!
```

```
=== UPDATED BILL AFTER REMOVAL ===
New Total Bill: PHP 870

=== CLEANING UP REMAINING ITEMS ===
[Destructor] Fruit: Apple destroyed.
[Destructor] GroceryItem base class cleaned up.
[Destructor] Fruit: Banana destroyed.
[Destructor] GroceryItem base class cleaned up.
[Destructor] Vegetable: Broccoli destroyed.
[Destructor] GroceryItem base class cleaned up.
PS C:\Users\TIPQC\Documents\Almirol joel cpes4>
```

Conclusion

Through the laboratory exercise conducted, it was clear that effective memory management within the C++ programming language depended on the comprehension of how compilers handle objects' lifecycles in both the stack and heap. Through the original and modified programs, there were indications of how silent memory leaks could take place; through the creation of ten Student objects but destroying only five of them, it was evident that objects created using new could get stuck in the heap if either value copying or failure to use the delete operator happens. This additional exercise presented a platform to solve this problem through the construction of a polymorphic array of GroceryItem pointers. Through the use of abstract base class with virtual destructor, it was clear that at runtime deallocation of objects inherited from base class (Fruit and Vegetable), not only will the memory allocated for the derived class be freed but also the memory allocation for the base class would be deleted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts,*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: to help understand the fundamental C++ concepts related to pointers, virtual destructors, and dynamic memory allocation. It helped in fixing syntax errors and properly structuring the code of the supplementary grocery list program. The use of AI did not substitute for my own reasoning, all the program logic, polymorphism design, observations, and analysis conclusions were made by me completely to guarantee academic integrity and all the code is i did it of code

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with

due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.